

Sistemas Operacionais 1

SINCRONIZAÇÃO DE PROCESSOS

Professor.: Dalton Matsuo Tavares

- Fundamentação teórica
- Problema da “seção crítica”
- Solução de Peterson
- Sincronização via Hardware
- Proteções via mutex (*Mutex Locks*)
- Semáforos
- Problemas clássicos de sincronização
- Monitores

- Fundamentação teórica
- Problema da “seção crítica”
- Solução de Peterson
- Sincronização via Hardware
- Proteções via mutex (*Mutex Locks*)
- Semáforos
- Problemas clássicos de sincronização
- Monitores

- Ferramenta de sincronização que oferece formas mais sofisticadas (do que *mutex locks*) para que um processo sincronize suas atividades.
- **Semaphore S** – variável inteira
- É um mecanismo de sinalização
- Permite que um número maior de threads (>1) acesse recursos compartilhados

Semáforo (cont.)

- Só pode ser acessado via duas operações indivisíveis (atômicas)
 - `wait()` e `signal()`
 - Originalmente chamado de `P()` e `V()`

- Definição da operação `wait()`

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

- Definição da operação `signal()`

```
signal(S) {  
    S++;  
}
```

- **Semáforo contador** – valor inteiro pode variar em um domínio de valores não restrito
 - Controla acesso a um recurso com um número finito de instâncias
- **Semáforo binário** – valor inteiro pode variar apenas entre 0 e 1
 - O mesmo que um **mutex lock**
- Pode resolver vários problemas de sincronização
- Considere P_1 e P_2 , os quais precisam que S_1 aconteça antes de S_2
 - Semáforo “**synch**” inicializado em 0

P1:

S_1 ;

`signal(synch);`

P2:

`wait(synch);`

S_2 ;

- Pode implementar um semáforo de contagem S como um semáforo binário

Implementação de Semáforo

- Deve garantir que dois processos não possam executar **wait()** e **signal()** no mesmo semáforo ao mesmo tempo
- A implementação se torna o problema da seção crítica, no qual o código do **wait** e **signal** são colocados na seção crítica
 - Pode ter **busy waiting** na implementação da seção crítica
 - Entretanto, o código de implementação é curto
 - Pouca espera ocupada se a seção crítica está raramente ocupada
- Observe que as aplicações podem gastar muito tempo em suas seções críticas e assim, esta não é uma boa solução

Implementação de Semáforo sem Busy waiting

- Com cada semáforo existe uma fila de espera associada
- Cada entrada na fila de espera possui dois parâmetros:
 - value (tipo inteiro)
 - Ponteiro para o próximo registro na lista
- Duas operações:
 - **block** – coloca o processo invocando a operação na fila de espera apropriada
 - **wakeup** – remove um processo na fila de espera e o coloca na fila de processos prontos

```
typedef struct{  
    int value;  
    struct process *list;  
} semaphore;
```


Implementação de Semáforo sem Busy waiting (Cont.)

```
wait(semaphore *S) { // Operação atômica
    S->value--;
    if (S->value < 0) { // Valores podem ser negativos
        adiciona o processo à S->list;
        block();
    }
}
```

```
signal(semaphore *S) { // Operação atômica
    S->value++;
    if (S->value >= 0) {
        remove o processo P da S->list;
        wakeup(P);
    }
}
```

Ex.:

- sem_ex1.c
- sem_ex2.c
- sem_ex3.c
- counting_sem.c
- prod_cons.c

Deadlock e Starvation

- **Deadlock** – dois ou mais processos estão aguardando indefinidamente por um evento que pode ser causado por apenas um dos processos em espera
 - Ex. Seja **S** e **Q** dois semáforos inicializados em 1

P_0

`wait(S) ;`

`wait(Q) ;`

`...`

`signal(S) ;`

`signal(Q) ;`

P_1

`wait(Q) ;`

`wait(S) ;`

`...`

`signal(Q) ;`

`signal(S) ;`

- **Ex. deadlock.c**

- **Starvation** – bloqueio indefinido
 - Um processo pode nunca ser removido da fila do semáforo na qual está suspenso
- **Inversão de prioridade** – problema de escalonamento que ocorre quando um processo de baixa prioridade mantém um lock necessário a processo de alta prioridade
 - Resolvido via **protocolo de herança de prioridade**
 - Ex. <https://users.cs.duke.edu/~carla/mars.html>

Problemas Clássicos de Sincronização

- Problemas clássicos usados para testar novos esquemas de sincronização propostos
- Problema do buffer limitado (Bounded-Buffer)
 - Problema dos leitores e escritores
 - Problema do jantar dos filósofos

Problema do Buffer Limitado

- n buffers, cada um pode manter um item
- Semáforo **mutex** inicializado para o valor 1
- Semáforo **full** inicializado para o valor 0
- Semáforo **empty** inicializado para o valor n

Problema do Buffer Limitado (Cont.)

- Estrutura do processo produtor

```
do {  
    ...  
    /* produz um item em next_produced */  
    ...  
    wait(empty);  
    wait(mutex);  
    ...  
    /* adiciona next_produced ao buffer */  
    ...  
    signal(mutex);  
    signal(full);  
} while (true);
```

Problema do Buffer Limitado (Cont.)

- Estrutura do processo consumidor

```
do {  
    wait(full);  
    wait(mutex);  
  
    ...  
    /* remove um item do buffer e coloca em next_consumed */  
    ...  
    signal(mutex);  
    signal(empty);  
  
    ...  
    /* consome o item em next_consumed */  
    ...  
} while (true);
```


Problema dos Leitores-Escritores

- Um conjunto de dados é compartilhado entre um certo número de processos concorrentes
 - Leitores – somente lêem o conjunto de dados; eles **não** realizam atualizações
 - Escritores – podem ler e escrever
- **Problema** – permitir que múltiplos leitores leiam ao mesmo tempo
 - Somente um escritor pode acessar o dado compartilhado em um dado momento
- Várias variações de como os leitores e escritores são considerados – todas envolvem algum tipo de prioridade
- Dados compartilhados
 - Conjunto de dados
 - Semáforo `rw_mutex` inicializado em 1
 - Semáforo `mutex` inicializado em 1
 - Inteiro `read_count` inicializado em 0

Problema dos Leitores-Escritores (Cont.)

- Estrutura do processo escritor

```
do {  
    wait(rw_mutex) ;  
    ...  
    /* Escrita realizada */  
    ...  
    signal(rw_mutex) ;  
} while (true) ;
```

Problema dos Leitores-Escritores (Cont.)

- Estrutura do processo leitor

- do {

```
    wait(mutex) ;
    read_count++;
    if (read_count == 1)
        wait(rw_mutex) ;

    signal(mutex) ;

    ...
    /* Leitura realizada */
    ...

    wait(mutex) ;
    read_count--;
    if (read_count == 0)
        signal(rw_mutex) ;

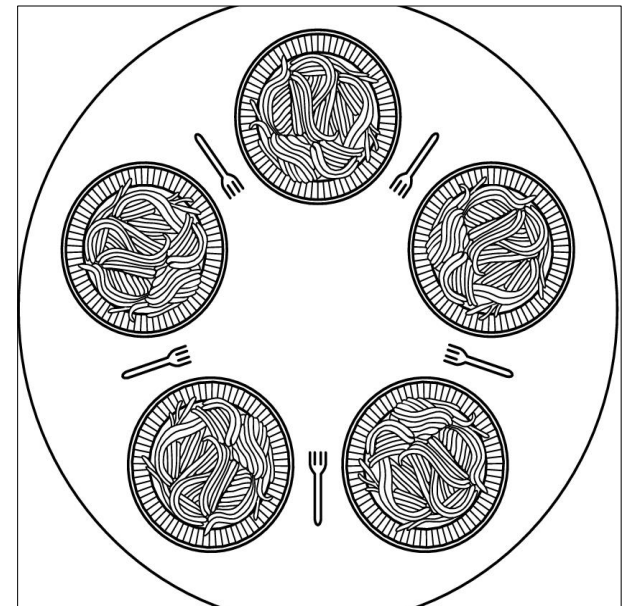
    signal(mutex) ;
} while (true) ;
```

Variações do Problema dos Leitores-Escritores

- ***Primeira*** variação – nenhum leitor é mantido em espera, a menos que o escritor tenha permissão para usar o objeto compartilhado
- ***Segunda*** variação – uma vez que o escritor esteja pronto, ele desempenha a escrita assim que possível.
- **Em ambos os casos pode haver *starvation*** levando a ainda mais variações
- Problema é solucionado em alguns sistemas, por meio de *reader-writer locks* fornecidos pelo kernel

Problema do Jantar dos Filósofos

- Premissa: filósofos gastam suas vidas pensando e comendo
- Não interagem com seus vizinhos, ocasionalmente tentam pegar dois garfos (um por vez) para comer
 - Precisam de dois garfos para comer, e então liberam ambos quando terminam
- No caso de 5 filósofos
 - Dados compartilhados
 - Garfos
 - Semáforo **N [5]** inicializado em 1



Algoritmo para o Problema do Jantar dos Filósofos

```
#define N 5                                /* número de filósofos */

void philosopher(int i)                    /* i: número do filósofo, de 0 a 4 */
{
    while (TRUE) {
        think( );                          /* o filósofo está pensando */
        take__fork(i);                      /* pega o garfo esquerdo */
        take__fork((i+1) % N);              /* pega o garfo direito; % é o operador modulo */
        eat( );                             /* hummm! Espaguetel */
        put__fork(i);                       /* devolve o garfo esquerdo à mesa */
        put__fork((i+1) % N);               /* devolve o garfo direito à mesa */
    }
}
```

□ Qual é o problema com esse algoritmo?

- Tratamento de Deadlock
 - Permitir que até 5 filósofos estejam sentados simultaneamente a mesa
 - Permitir que um filósofo pegue os garfos somente se ambos estiverem disponíveis (isso deve ser feito em uma seção crítica).
 - **Possibilidade:** um filósofo numerado pega primeiro o garfo da esquerda e então, o da direita mediante a verificação do estado de seus vizinhos.

Algoritmo para o Problema do Jantar dos Filósofos (Cont.)

```
#define N          5          /* número de filósofos */
#define LEFT      (i+N-1)%N    /* número do vizinho à esquerda de i */
#define RIGHT     (i+1)%N     /* número do vizinho à direita de i */
#define THINKING  0          /* o filósofo está pensando */
#define HUNGRY    1          /* o filósofo está tentando pegar garfos */
#define EATING    2          /* o filósofo está comendo */
typedef int semaphore;        /* semáforos são um tipo especial de int */
int state[N];                /* arranjo para controlar o estado de cada um */
semaphore mutex = 1;         /* exclusão mútua para as regiões críticas */
semaphore s[N];              /* um semáforo por filósofo */

void philosopher(int i)      /* i: o número do filósofo, de 0 a N-1 */
{
    while (TRUE) {           /* repete para sempre */
        think( );            /* o filósofo está pensando */
        take_forks(i);        /* pega dois garfos ou bloqueia */
        eat( );               /* hummm! Espaguete! */
        put_forks(i);         /* devolve os dois garfos à mesa */
    }
}
```


Algoritmo para o Problema do Jantar dos Filósofos (Cont.)

```
void take_forks(int i)                /* i: o número do filósofo, de 0 a N-1 */
{
    down(&mutex);                     /* entra na região crítica */
    state[i] = HUNGRY;                /* registra que o filósofo está faminto */
    test(i);                          /* tenta pegar dois garfos */
    up(&mutex);                       /* sai da região crítica */
    down(&s[i]);                      /* bloqueia se os garfos não foram pegos */
}

void put_forks(i)                    /* i: o número do filósofo, de 0 a N-1 */
{
    down(&mutex);                     /* entra na região crítica */
    state[i] = THINKING;              /* o filósofo acabou de comer */
    test(LEFT);                      /* vê se o vizinho da esquerda pode comer agora */
    test(RIGHT);                     /* vê se o vizinho da direita pode comer agora */
    up(&mutex);                       /* sai da região crítica */
}

void test(i)                         /* i: o número do filósofo, de 0 a N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

- Uso incorreto de operações envolvendo semáforos:
 - `signal (mutex) ... wait (mutex)`
 - `wait (mutex) ... wait (mutex)`
 - Omissão de `wait (mutex)` ou `signal (mutex)` (ou ambos)
- *Deadlock e starvation* se tornam possíveis.

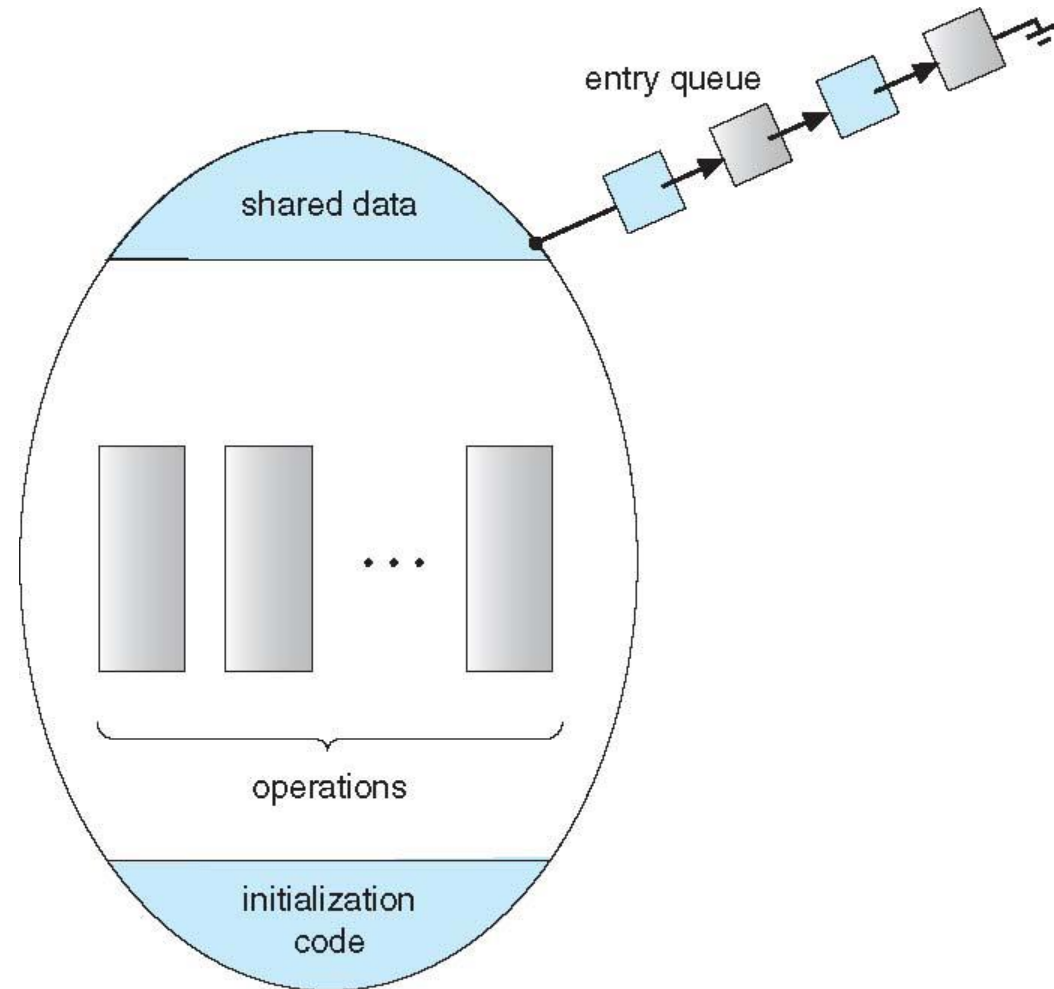
- Uma abstração de alto nível que oferece um mecanismo conveniente e efetivo para a sincronização de processos
- É um *tipo abstrato de dado* (TAD), no qual variáveis internas são acessíveis somente por código interno ao procedimento
- Somente um processo pode estar ativo dentro do monitor em um dado momento
- Não é “poderoso” o suficiente para modelar alguns esquemas de sincronização
 - Necessário definir mecanismos de sincronização adicionais (construto *condition*)
 - Um programador que precisa criar mecanismos de sincronização particulares, pode definí-los em uma ou mais variáveis do tipo *condition*

```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { ... }

    procedure Pn (...) {...}

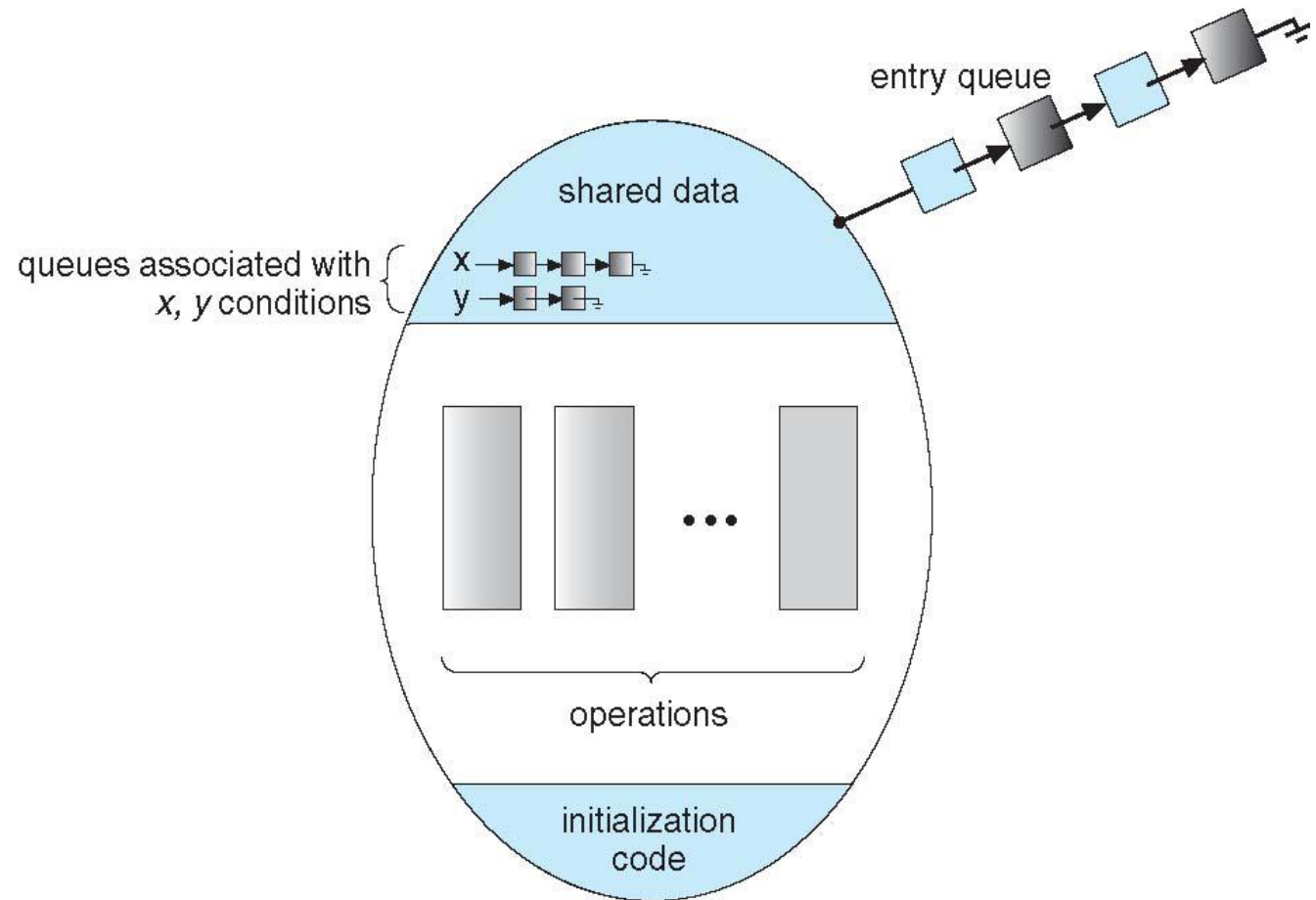
    Initialization code (...) { ... }
}
}
```

Visão Esquemática de um Monitor



- `condition x, y;`
- Duas operações são permitidas em uma variável de condição:
 - `x.wait()` – um processo que invoca a operação é suspenso até a execução de uma operação do tipo `x.signal()`
 - `x.signal()` – permite que um dos processos continue (caso exista algum) que invocou `x.wait()`
 - Se nenhum `x.wait()` foi executado na variável, então ele não tem efeito sobre a variável

Monitor com Variáveis de Condição



Escolhas de Variáveis de Condição

- Se o processo P invoca **`x.signal()`**, e o processo Q é suspenso em **`x.wait()`**, o que acontece em seguida?
 - Ambos Q e P não podem executar em paralelo, Se Q é continuado, então P deve esperar
- Opções incluem
 - **signal e wait** – P espera até que Q deixe o monitor ou ele espera por outra condição
 - **signal and continue** – Q espera até P deixar o monitor ou ele espera por outra condição
 - Ambos tem prós e contras – a própria implementação da linguagem pode decidir
 - Monitores implementados em Pascal Concorrente fazem uma escolha
 - P executando **signal** imediatamente deixa o monitor, Q é continuado
 - Implementado em outras linguagens incluindo Mesa, C#, Java
 - Em C, por exemplo: `pthread_cond_t` para declarar variáveis de condição em *threads*.
 - http://pubs.opengroup.org/onlinepubs/7908799/xsh/pthread_cond_wait.html

Ex1. Solução para o Jantar dos Filósofos

- Cada filósofo i invoca as operações **pickup()** e **putdown()** na seguinte sequência:

`DiningPhilosophers.pickup(i) ;`

EAT

`DiningPhilosophers.putdown(i) ;`

- Não ocorre *deadlock* (*starvation* é possível)

Ex1. Solução para o Jantar dos Filósofos (Cont.)

monitor DiningPhilosophers

```
{
    enum {THINKING; HUNGRY, EATING} state [5];
    condition self [5];

    void pickup (int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING) self[i].wait;
    }

    void putdown (int i) {
        state[i] = THINKING;
        // testa vizinhos da esquerda e direita
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}
```

Ex1. Solução para o Jantar dos Filósofos (Cont.)

```
void test (int i) {  
    if ((state[(i + 4) % 5] != EATING) &&  
        (state[i] == HUNGRY) && (state[(i + 1) % 5] != EATING) ) {  
        state[i] = EATING ;  
        self[i].signal () ;  
    }  
}  
  
initialization_code() {  
    for (int i = 0; i < 5; i++)  
        state[i] = THINKING;  
}
```

Continuando Processos em um Monitor

- Caso existam vários processos em fila na condição x , e $x.\text{signal}()$ é executado, qual deve continuar?
 - FCFS frequentemente não é adequado
- **Espera condicional (*conditional-wait*)** na forma $x.\text{wait}(c)$
 - Onde c é **um número de prioridade**
 - Processo com menor número (maior prioridade) é escalonado em seguida

Alocação de Um Único Recurso

- Aloque um único recurso entre processos concorrentes usando números de prioridade que especifiquem o tempo máximo para o qual um processo planeja usar o recurso

```
R.acquire(t) ;
```

```
...
```

```
access the resource;
```

```
...
```

```
R.release;
```

- Onde R é uma instância do tipo **ResourceAllocator**

Monitor para Alocar um Único Recurso

```
monitor ResourceAllocator
{
    boolean busy;
    condition x;
    void acquire(int time) {
        if (busy)
            x.wait(time);
        busy = TRUE;
    }
    void release() {
        busy = FALSE;
        x.signal();
    }
    initialization code() {
        busy = FALSE;
    }
}
```